Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Background task kavramı
  - Arka plan işlerinin tanımı: kullanıcı ile doğrudan etkileşimde olmayan, bir arayüz ile bağlantısı olmayan, genelde uzun süren (milisaniyelerin ötesinde) ve herhangi bir uygulamanın bir aktivitesi tarafından başlatılmamış işler.
  - Arka plan işlerinin neden gerekli olduğu: kullanıcı etkileşiminin kesintiye uğramaması, kaynakların verimli kullanımı, performans artırma.
  - Ana thread (main thread, UI thread) bütün UI bileşenlerini ve kullanıcı ilişkisini yürütür; uzun süreli işler burada yapılırsa ekran donar (ANR - Application Not Responding).
- Arka plan iş yönetim yapıları
  - Handler: tek bir thread'in yönetiminden sorumlu; veri tabanı bağlantısı, tek bir veri sorgusu gibi tek işler için uygun.
  - ExecutorService: birden fazla thread'in yönetiminden sorumlu; thread'lerle ilgili genel bilgi almayı sağlar (thread'in ne kadar çalıştığı, ne kadar beklediği); thread pool oluşturarak birden fazla thread'i yönetir.
  - Broadcast Receiver: manifestte register edildiğinde tipik bir arka plan işi; sistem koşulu oluştuğunda onReceive metodu çalışır.
  - AlarmManager: belirli işleri belirlenen saatlerde bir kez veya tekrar edecek şekilde (saatlik, günlük, haftalık) çalıştırır; daha strikt, kaynak tüketimi yüksek.
  - WorkManager: JobScheduler ile uyumlu çalışan modern API; belli koşullar (güç bağlantısı, network) oluştuğunda işlerin çalışmasını sağlar; foreground service olarak da belli işlerin çalışmasından sorumlu olabilir; kaynak tüketimi açısından daha verimli.
  - Foreground Service: kullanıcı arayüzüne sahip olmayan, uzun süre kullanıcı ile interaksiyon olmadan çalışan yapı; notification tray, pop-up dialog box, voice command gibi yapılarla iletişim; en öncelikli, en korunan, sistem tarafından ortadan kaldırılma olasılığı çok düşük işler.
- Background task kategorizasyonu
  - Üç temel kategori: hemen yapılması gerekenler (immediate), belirlenmiş kesin zamanda yapılması gerekenler (exact), belli bir periyoda tamamlanması gereken veya ertelenebilir olanlar (deferred).
  - Hangi yapının seçileceği şu sorulara verilen cevaba göre belirlenir:
    - İş kesin bir saatte mi çalışacak? (AlarmManager)
    - Hemen mi yapılması gerekiyor? (WorkManager veya Foreground Service)
    - Belli periodlarla mı çalışacak? (WorkManager)
    - İş kesintiye uğrayabilir mi? (büyük upload/download için dikkat)
    - Cihaz koşullarıyla ilişkisi var mı? (güç kaynağı, Wi-Fi)
    - Hassas kullanıcı verisi toplamayı içeriyor mu? (lokasyon bilgisi vb.)
- Android background task yönetiminin tarihsel gelişimi
  - Android 6.0 öncesi: inanılmaz özgürlük; istenen iş arka plana atılıyor, sınırsız çalışabiliyor; RAM ve pil tüketimi olumsuz etkileniyor; ANR hataları.
  - Android 6.0 (Doze): pil tüketimi için çözüm; telefon kullanılmıyorsa ve ekran kapalıysa sabit olarak hareket etmediğini tespit edip arka plan sıklığını azaltır.
  - Android 7 (Doze on the Go): hareket halinde de arka plan servislerini yönetir; araçla seyahat gibi durumları tespit eder.
  - Android 9 (App Standby Buckets): uygulamaları kullanım karakteristiklerine göre

önceliklendirir; makine öğrenmesi yaklaşımı.
- App Standby Buckets
  - 5 kategori (bucket):
    - Active: şu anda kullanılan veya çok yakında kullanılmış uygulamalar.
    - Working set: gün içinde sık kullanılan (sosyal medya gibi).
    - Frequent: günde bir iki kez veya gün aşırı kullanılan (gym uygulaması gibi).
    - Rare: ayda yılda bir kez kullanılan (seyahat uygulamaları gibi).
    - Never: hiç uğranmamış uygulamalar.
  - Sistem karar verir, makine öğrenmesi kullanır, müdahale edilmemelidir.
  - Amaç: sık kullanılan uygulamalara daha fazla kaynak (CPU, RAM, pil) ayırmak; nadir kullanılanları kısıtlamak.
  - Launcher olmadan uygulama hiçbir zaman Active bucket'a giremez.
  - Foreground service'e sahip uygulamalar Active olabilir (müzik uygulaması gibi).
  - Content provider üzerinden veri senkronizasyonu yapan uygulamalar da Active'te.
  - getAppStandByBucket metodu ile bucket durumu gözlemlenebilir (UsageStatsManager).
- Foreground Service kullanım örnekleri
  - Müzik uygulaması: arka planda çalıp notification bar üzerinden kontrol.
  - Sesli/görüntülü konuşma uygulamaları: kesintisiz hizmet kalitesi.
  - Navigasyon uygulamaları: arka planda lokasyon takibi + sesli direktifler.
  - Download uygulamaları: indirme sırasında kesinti olmaması (ama Download Manager yapısı da var).
- AlarmManager
  - Belirli zamanlarda çalışacak işler için: tek bir kez gerçekleşebilir veya birden fazla (saatlik, günlük, haftalık).
  - Örnek: gece 2'de yedekleme, sabah 7'de alarm.
  - WorkManager'ın AlarmManager'a göre avantajı: kaynak tüketimi açısından daha verimli (esneklik var).
  - AlarmManager kesin dakika/saniyede çalışmak zorunda olduğu için kaynak tüketimi yüksek.
- Service (Servis) bileşeni
  - Android'in dört temel bileşeninden biri.
  - Kullanıcı arayüzüne sahip olmayan, uzun süre kullanıcı ile interaksiyon olmadan çalışan yapı.
  - Günümüzde en son başvurulan komponentlerden biri (alternatif çözümler çıkmasıyla).
  - Örnek: sağlık bilgilerini arka planda takip eden uygulama; GPS, accelerometer, gyroscope, magnetometer gibi sensörlerden faydalanır.
- Doze modunun detayları
  - Telefon hareketsiz kaldığında arka plan lokasyon bilgisi sıklığını azaltır; sıklık saniyede 1'den yarım dakika/dakikada 1'e çıkabilir.
  - 7 ile gelen Doze on the Go: araç hareket halindeyken de (stationer pozisyonda değilken) arka plan servislerini yönetir; kaynak tasarrufu sağlar.
  - Android 9'da uygulamaların kullanım sıklığına göre bucket'lara ayrılması (App Standby Buckets).

Hocanın Özellikle Vurguladığı Kısımlar

- Doze modunun yazılımsal çözüm olarak kritikliği
  - Android 6.0 ile gelen Doze modunun yazılımsal bir çözüm olduğu, pil tüketimini azaltmak için telefonun hareketsizliğini tespit edip arka plan sıklığını azalttığı.
  - "Yazılımsal bir çözüm olduğunu hatırlıyorum" diyen öğrenciye hoca bu tespiti onaylamıştır.
- Background işlerde main thread'den kaçınmanın önemi
  - "Application not responding" hatasının en önemli sebebinin kısa sürmeyen işlerin main thread'de yapılması.
  - User experience'ın kötü deneyimler yaşatmaması için hangi yapıların kullanılacağının iyi bilinmesi.
- App Standby Buckets'e müdahale edilmemesi gerektiği
  - "Kesinlikle müdahale edilmemesi gerekiyor" vurgusu; sistem bu yapıyı makine öğren-

mesi ile yönetir.
    - Bucket'lar zaman içinde değişebilir; uygulama bir bucket ile girdiğinde hep orada kalmaz.
    - Üreticinin device'e bağlı olarak farklı aritmetikle çalıştığı, dinamik bir yapı olduğu.
- Foreground service'in kritik senaryolar için kullanılması
    - Müzik, video konferans, navigasyon gibi kesintisiz çalışması gereken uygulamalar.
    - Sistem tarafından ortadan kaldırılma olasılığı en düşük yapı olduğu.
    - Download manager'ın Android'in sunduğu bir başka yapı olduğu, download için foreground service yerine onun da tercih edilebileceği.
- Sık kullanılan uygulamalara daha fazla kaynak ayrılmasının amacı
    - "Hedef pil tüketimini minimize etmek"; RAM'i az kullanmak değil, pili az kullanmak asıl amaç.
    - "Daha çok RAM'i az kullanmak değil onları az kullandığınızda doğal olarak pili de az kullanmış oluyorsunuz" ifadesi.

Kısa Tekrar Notları

- Background task = kullanıcı etkileşimi olmayan, UI'sız, uzun süreli iş.
- 3 kategori: immediate, exact time, deferred.
- Handler (tek thread) vs ExecutorService (çoklu thread, thread pool).
- AlarmManager: kesin zamanda; WorkManager: esnek; ForegroundService: kesintisiz.
- Android background yönetimi: 6.0 Doze → 7 Doze on the Go → 9 App Standby Buckets.
- 5 bucket: Active, Working set, Frequent, Rare, Never.
- getAppStandByBucket: bucket durumunu öğrenmek için (UsageStatsManager).
- Müdahale etmeyin, sistem karar verir.
- Foreground service en korunan background yapısı.
- Service: kullanıcı arayüzü olmayan, uzun süreli bileşen (artık son tercih).
- Tüm background yapılarının asıl amacı: pil tüketimini minimize etmek.

Detaylı Açıklamalar

Dersin başlangıcında hoca, derse 30 dakika geç başlamak zorunda kaldığını belirterek özür dilemiştir. Bu hafta background task'ları konuşacakları, Android tarafında arka plan işlerini yönetmek için çeşitli seçenekler olduğu belirtilmiştir. Küçük bir uygulama örneği verileceği, arkasından notification'lar anlatılacağı ve haftanın noktalanacağı söylenmiştir. Bir sonraki hafta location based servislerden, haritalardan bahsedileceği ve dönemin tamamlanacağı belirtilmiştir. Dönem projesi ile ilgili soru sorulmuş, ancak spesifik bir soru gelmemiştir.

Background task kavramı detaylı olarak açıklanmıştır. Arka plan işlerinin tanımı öğrencilerle etkileşimli olarak yapılmıştır. Bir öğrenci web servisi ile yedekleme veya arka planda dosya indirme örneği vermiştir. Hoca bunu kabul etmiş, lokasyon bilgisini web servisine gönderme örneğini de eklemiştir. Resmi olarak şu özellikler tanımlanmıştır: kullanıcı ile etkileşimde olmayan işler, bir arayüz ile bağlantısı olmayan işler, uzun süren (milisaniyelerin ötesinde, saniyenin üzerinde) işler, herhangi bir uygulamanın bir aktivitesi tarafından başlatılmamış foreground servisler. Ana thread (main thread, UI thread) bütün UI bileşenlerini ve kullanıcı ilişkisini yürütür; uzun süreli işler burada yapılırsa ekran donar (freeze eder, dolar, bloklanır) ve bu problem olur.

Arka plan iş yönetim yapıları detaylı olarak ele alınmıştır. Handler tek bir thread'in yönetiminden sorumlu; veri tabanı bağlantısı, tek bir veri sorgusu gibi tek işler için uygun. ExecutorService birden fazla thread'in yönetiminden sorumlu; thread'lerle ilgili genel bilgi almayı sağlar (thread'in ne kadar çalıştığı, ne kadar beklediği); thread pool oluşturarak birden fazla thread'i yönetir. Bir öğrencinin Java'daki Executor sorusu üzerine hoca, Android'e spesifik bir yapı olmadığını, aynı yapının burada da kullanıldığını belirtmiştir. Broadcast Receiver manifestte register edildiğinde tipik bir arka plan işi; sistem koşulu oluştuğunda onReceive metodu çalışır. AlarmManager belirli işleri belirlenen saatlerde bir kez veya tekrar edecek şekilde (saatlik, günlük, haftalık) çalıştırır; daha strikt, kaynak tüketimi yüksek. WorkManager JobScheduler ile uyumlu çalışan modern API; belli koşullar (güç bağlantısı, network) oluştuğunda işlerin çalışmasını sağlar; foreground service olarak da belli işlerin çalışmasından sorumlu olabilir; kaynak tüketimi açısından AlarmManager'a göre daha verimli. Foreground Service kullanıcı

arayüzüne sahip olmayan, uzun süre kullanıcı ile interaksiyon olmadan çalışan yapı; notification tray, pop-up dialog box, voice command gibi yapılarla iletişim; en öncelikli, en korunan, sistem tarafından ortadan kaldırılma olasılığı çok düşük işler.

Background task kategorizasyonu üç temel kategori üzerinden açıklanmıştır. Hemen yapılması gerekenler (immediate), belirlenmiş kesin zamanda yapılması gerekenler (exact), belli bir periyoda tamamlanması gereken veya ertelenebilir olanlar (deferred). Hangi yapının seçileceği şu sorulara verilen cevaba göre belirlenir: İş kesin bir saatte mi çalışacak? Hemen mi yapılması gerekiyor? Belli periodlarla mı çalışacak? İş kesintiye uğrayabilir mi? (büyük upload/download için dikkat, Wi-Fi üzerine indirilmediğinde gigabyte'ların boşa kullanılması). Cihaz koşullarıyla ilişkisi var mı? (güç kaynağı, Wi-Fi). Hassas kullanıcı verisi toplamayı içeriyor mu? (lokasyon bilgisi).

Android background task yönetiminin tarihsel gelişimi detaylı olarak anlatılmıştır. Android 6.0 öncesi inanılmaz bir özgürlük vardı; istenen iş arka plana atılıyordu, arka plan sınırsız bir özgürlükle processleri çalıştırabiliyordu. Bu da özellikle RAM ve pil tüketimini çok olumsuz etkiliyordu. Birçok uygulamanın farkında olmadan pillerin hızla bitmesine ve ANR (Application Not Responding) hatalarına yol açıyordu. 6.0 ile birlikte Android yeni bir düzen oluşturmaya başladı: Doze modu devreye sokuldu. Bir öğrencinin "yazılımsal bir çözüm olduğunu hatırlıyorum" tespiti hoca tarafından onaylanmıştır. Doze, telefonun hareketsiz kaldığı anları tespit edip arka plan sorgulamalarını azaltır; sıklık saniyede 1'den yarım dakika/dakikada 1'e çıkabilir. 7 ile birlikte Doze on the Go geldi: araçla seyahat gibi durumları tespit edip arka plan servislerini yönetir. 9'da App Standby Buckets geldi: uygulamaları kullanım karakteristiklerine göre önceliklendirir.

App Standby Buckets 5 kategoriden oluşur: Active (şu anda kullanılan veya çok yakında kullanılmış uygulamalar), Working set (gün içinde sık kullanılan, sosyal medya gibi), Frequent (günde bir iki kez veya gün aşırı kullanılan, gym uygulaması gibi), Rare (ayda yılda bir kez kullanılan, seyahat uygulamaları gibi), Never (hiç uğranmamış uygulamalar). Sistem karar verir, makine öğrenmesi kullanır; müdahale edilmemelidir. Bucket'lar zaman içinde değişebilir; uygulama bir bucket ile girdiğinde hep orada kalmaz. getAppStandByBucket metodu ile bucket durumu UsageStatsManager üzerinden gözlemlenebilir. Hoca, "sistemin bunu kendi yönettiğini ve her bir üreticinin ürettiği device'e bağlı olarak farklı bir aritmetikle çalıştığını rahatlıkla söyleyebiliriz" diyerek dinamik yapıyı vurgulamıştır. Active bucket'a girmek için launcher olması şarttır; foreground service'e sahip uygulamalar (müzik uygulaması gibi) Active olabilir; content provider üzerinden veri senkronizasyonu yapan uygulamalar da Active'tedir. Hoca, "daha sık kullanılan uygulamalara daha fazla öncelik vererek kullanıcının günlük telefon kullanımını rahatlatmak" diyen öğrenciye, "daha fazla kaynak temin ederek" şeklinde düzeltmiştir: "Ne kadar önceliği yüksekse o uygulama daha fazla kaynak alıyor. Bütün aslında mantık bu." Hoca ayrıca "temel hedef enerji tüketimini minimize etmek" vurgusunu yapmıştır.

Foreground Service kullanım örnekleri açıklanmıştır. Müzik uygulaması: arka planda çalıp notification bar üzerinden kontrol. Sesli/görüntülü konuşma uygulamaları: kesintisiz hizmet kalitesi. Navigasyon uygulamaları: arka planda lokasyon takibi + sesli direktifler. Download uygulamaları için bir öğrenci download'ın foreground service olup olmadığını sormuş, hoca "Android'in sunduğu bir başka yapı daha var, download manager" diyerek alternatif sunmuştur. Hoca ayrıca "en öncelikli, en korunan ve genelde sistem tarafından ortadan kaldırılma olasılığı çok düşük olan işler" diyerek foreground service'in önemini vurgulamıştır.

AlarmManager ve WorkManager karşılaştırması yapılmıştır. AlarmManager belirli zamanlarda çalışacak işler için: tek bir kez gerçekleşebilir veya birden fazla (saatlik, günlük, haftalık). Örnek: gece 2'de yedekleme, sabah 7'de alarm. WorkManager'ın AlarmManager'a göre avantajı: kaynak tüketimi açısından daha verimli (esneklik var). AlarmManager kesin dakika/saniyede çalışmak zorunda olduğu için kaynak tüketimi yüksek. Hoca, "Work Manager özellikle Job Scheduler'la da uyumlu bir şekilde çalışıyor. Yeni versiyonu olarak da düşünebilirsiniz" demiştir.

Service bileşeni kısaca açıklanmıştır. Android'in dört temel bileşeninden biri (activity, service, broadcast receiver, content provider). Kullanıcı arayüzüne sahip olmayan, uzun süre kullanıcı ile interaksiyon olmadan çalışan yapı. Günümüzde en son başvurulan komponentlerden biri (alternatif çözümler çıkmasıyla). Örnek: sağlık bilgilerini arka planda takip eden uygulama; GPS, accelerometer, gyroscope, magnetometer gibi sensörlerden faydalanır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.